Sparsification Operators for Sparse Embedding Compression

Tomáš Novotný
Faculty of Information Technology, CTU in Prague
Prague, Czech Republic
novott37@fit.cvut.cz

ABSTRACT

High-dimensional embedding tables in modern recommendation and retrieval systems impose significant memory footprints and latency overheads. Sparse Autoencoders (SAEs) provide a data-driven compression technique by learning sparse latent representations. In this work, we empirically evaluate three sparsification operators within the CompresSAE framework: the baseline Standard per-sample TopK, a flexible Batch TopK enforcing average sparsity, and a gradient-based Gated TopK using learned gating with straight-through estimation. Using MovieLens embeddings encoded by nomic-embed-text-v1.5 and reduced to 256 dimensions, we assess reconstruction fidelity and retrieval Recall@100 across various sparsity budgets. Our results demonstrate that Batch TopK offers a robust alternative, achieving retrieval performance comparable to or exceeding the baseline at lower inference latencies, but at the cost of increased training time, whereas the gradient-based approach suffers from training instability.

KEYWORDS

Embedding compression, sparse autoencoders, sparsification operators

1 INTRODUCTION

High-dimensional embeddings are the backbone of modern recommender systems, information retrieval, and natural language processing applications. They allow for the semantic representation of users, items, and queries in a continuous vector space. However, as the cardinality of these entities grows into the millions or billions, the resulting embedding tables become a primary bottleneck, consuming vast amounts of memory and inducing significant latency during retrieval operations.

To mitigate these issues, embedding compression has become a critical area of research. Traditional techniques such as dimensionality reduction (e.g., PCA) often discard non-linear information, while quantization methods (e.g., Product Quantization) introduce approximation errors that can degrade retrieval precision. Recently, Sparse Autoencoders (SAEs) have emerged as a promising alternative, capable of learning high-dimensional sparse representations that are both storage-efficient and effective for retrieval. The CompresSAE framework demonstrated the viability of this approach using a standard TopK sparsification operator.

However, the standard TopK operator imposes a rigid constraint by enforcing a fixed number of active latent features for every input, regardless of its information content. This lack of flexibility may lead to suboptimal resource allocation. In this paper, we investigate whether alternative sparsification operators can enhance the CompresSAE framework. Specifically, we examine if the rigid per-sample TopK operator can be replaced with mechanisms that allow for more flexible sparsity allocation. We further analyze how different sparsification operators impact reconstruction quality and downstream retrieval performance, and determine if there exists an operator that outperforms the standard baseline in terms of fidelity, accuracy, and practical efficiency.

We address these inquiries by comparing the Standard TopK baseline against a Batch TopK operator, which enforces sparsity constraints at the batch level, and a Gated TopK operator, which learns to select features via a differentiable gating mechanism.

2 BACKGROUND AND RELATED WORK

Modern recommender and retrieval systems rely heavily on high-dimensional embedding tables to represent users, items, and queries. As the cardinality of entities grows—often reaching millions or billions in industrial applications—these embedding tables become the primary bottleneck for memory consumption and inference latency [8, 10, 13]. The challenge is twofold — storing massive tables in GPU/TPU memory during training and deployment, and performing efficient Approximate Nearest Neighbor (ANN) search over these dense high-dimensional vectors. To address these constraints, a variety of embedding compression techniques have been proposed, which can be broadly categorized into dimensionality reduction, quantization, and weight sharing.

Traditional linear methods like Principal Component Analysis (PCA) reduce the embedding dimension but often lead to significant information loss in non-linear manifolds. Hash-based methods [3] and compositional codes replace dense vectors with compact binary codes, offering storage efficiency but frequently suffering from collision-induced noise that degrades retrieval precision.

Standard search methods like Vector Quantization (VQ) and Product Quantization (PQ) [5] decompose vectors into subspaces and approximate them using learned codebooks. While highly effective for storage, PQ-based distances are approximations that can limit Recall@K performance compared to exact search. More recently, low-precision training (e.g., INT8 or binary embeddings) has gained traction, though it requires specialized hardware support to fully realize latency gains [13].

Sparse Autoencoders (SAEs) have recently emerged as a powerful alternative, learning high-dimensional, sparse representations. Foundational work by Makhzani and Frey introduced the *k-Sparse Autoencoder*, which enforces sparsity by strictly retaining only the k highest activations in the latent layer [9].

Building on this, the CompresSAE framework [6] demonstrated that SAEs with a standard per-sample TopK operator [9] can outperform state-of-the-art methods like Matryoshka [7] in retrieval at industrial compression rates. However, the Standard TopK operator imposes a rigid constraint, requiring every sample to use exactly k active latents regardless of the input's information density.

This limitation has spurred research into more flexible sparsification operators. In the domain of Large Language Model (LLM)

interpretability, *Batch TopK* operator was proposed, which enforces an average sparsity of k across a batch rather than per sample [2]. This allows the model to adaptively allocate more latent resources to complex inputs and fewer to simple ones, though its utility for embedding compression remains under-explored.

Alternatively, gradient-based gating operators, such as Straight-Through Estimators (STE) [1], attempt to learn binary masks via gradient descent. While theoretically appealing, these methods frequently suffer from training instability and *dead neurons*, a problem recently addressed in generative models [11].

Our work compares the Standard TopK operator with Batch TopK, and Gated TopK operator specifically within the CompresSAE architecture. Unlike previous studies focused on interpretability, we assess these operators on their ability to preserve reconstruction quality and retrieval recall.

3 APPROACH & METHODS

Our approach builds upon the CompresSAE framework [6], which leverages an encoder-decoder architecture to learn sparse high-dimensional latent representations of input data (dense embeddings). The advantage of this framework is that it does not require finetuning of the encoder, allowing the use of pretrained models for initial dense embeddings.

Given the N data points, a pretrained encoder produces a corpus of dense embeddings $\mathbf{E} \in \mathbb{R}^{N \times h}$ of dimension h. Each embedding $\mathbf{x} \in \mathbb{R}^h$ is normalized $\bar{\mathbf{x}} = \mathbf{x}/\|\mathbf{x}\|_2$, processed individually through the SAE and encoded into a sparse latent vector $\mathbf{s} \in \mathbb{R}^d$ of latent dimension d.

$$\mathbf{s} = \phi(\mathbf{z}), \quad \mathbf{z} = \mathbf{W}_{enc}\bar{\mathbf{x}} + \mathbf{b}_{enc}, \quad \mathbf{W}_{enc} \in \mathbb{R}^{d \times h}, \ \mathbf{b}_{enc} \in \mathbb{R}^d.$$

In contrast to the original CompresSAE, encoder normalization was added, to prevent feature collapse.

A sparsification operator $\phi : \mathbb{R}^d \to \mathbb{R}^d$ produces sparse codes and serves as nonlinear activation.

The reconstruction $\hat{\mathbf{x}} \in \mathbb{R}^h$ is obtained by decoding the sparse latent representation to the original embedding space.

$$\hat{\mathbf{x}} = \mathbf{W}_{dec}\mathbf{s}, \quad \mathbf{W}_{dec} \in \mathbb{R}^{h \times d}.$$

The core innovation lies in the comparison of distinct sparsification operators, illustrated in Figure 1, that determine which latent features are retained for each vector. The encoder architecture slightly varies based on the sparsification operator used.

The important note is that all operators produce k (in-average in case of Batch TopK) non-zero entries per sample during inference. This ensures a predictable memory footprint and keeps the efficiency of the latent retrieval and the applicability of the kernel trick in reconstruction retrieval, presented in the original paper [6].

Standard TopK (Baseline). The original CompresSAE model employs a standard per-sample TopK operator to enforce exact sparsity of k non-zero entries based on the absolute magnitude of latent activations.

For a latent vector $\mathbf{z} \in \mathbb{R}^d$, let $\mathcal{I}_k(\mathbf{z}) \subset \{1, \ldots, d\}$ be the set of indices corresponding to the k largest absolute values in $\mathbf{z}$. The sparse code $\mathbf{s}$ is obtained by masking

$$\mathbf{s}_i = \begin{cases} \mathbf{z}_i & \text{if } i \in \mathcal{I}_k(\mathbf{z}), \\ 0 & \text{otherwise.} \end{cases}$$

Batch TopK.. To allow for variable information density while maintaining a global sparsity budget, we propose using the Batch TopK operator.

During training, given a batch $\mathbf{Z} \in \mathbb{R}^{n \times d}$, we define a global budget $K_{batch} = n \times k$. We identify the set of indices $\mathcal{J}$ corresponding to the top K_{batch} elements in the flattened absolute batch $|\mathbf{Z}|$. The binary mask $\mathbf{M} \in \{0, 1\}^{n \times d}$ is defined as $M_{ij} = 1 \iff (i, j) \in \mathcal{J}$. The sparsified batch is then $\mathbf{S} = \mathbf{M} \odot \mathbf{Z}$.

However, this mechanism requires batch-level statistics unavailable during single-item inference. To enable deployment, we calibrate a fixed threshold Γ_{fixed} on the validation set by collecting all activation magnitudituds $|z_{ij}|$ and selecting the value that corresponds to the $(1 - \frac{k}{d})$-th quantile of the empirical distribution, where d is the embedding dimension. This threshold is then applied element-wise during inference,

$$\mathbf{s}_i = \begin{cases} \mathbf{z}_i & \text{if } |\mathbf{z}_i| \geq \gamma, \\ 0 & \text{otherwise.} \end{cases}$$

This calibration procedure ensures that the expected sparsity budget during inference matches the target k under the i.i.d. assumption. The inference is generally faster than per-sample TopK, as it avoids sorting and selection operations at inference time.

Gated TopK (gradient-based). Standard magnitude-based pruning assumes that activation magnitude equals feature importance. To challenge this, we investigate gradient-based mechanism. A parallel gating network computes importance scores $\mathbf{g} = \sigma(\mathbf{W}_g\bar{\mathbf{x}} + \mathbf{b}_g)$ directly from the normalized input $\bar{\mathbf{x}}$, where σ is the sigmoid function and $\mathbf{W}_g \in \mathbb{R}^{d \times h}, \mathbf{b}_g \in \mathbb{R}^d$ are learnable parameters. Selection is performed based on the gate values $\mathbf{g}$ instead of the latent magnitudes $\mathbf{z}$. Let $\mathcal{I}_k(\mathbf{g})$ be the indices of the top-k elements in $\mathbf{g}$. The output is:

$$\mathbf{s}_i = \begin{cases} \mathbf{z}_i & \text{if } i \in \mathcal{I}_k(\mathbf{g}), \\ 0 & \text{otherwise.} \end{cases}$$

To enable training through the discrete selection step, we employ the Straight-Through Estimator (STE) [1]. In the backward pass, gradients with respect to the mask are approximated as identity for selected indices. The inference procedure remains similar to TopK, selecting the top-k gate indices, with higher computational overhead due to the additional gating network.

4 EXPERIMENTS AND RESULTS

To obtain embeddings corpus, we utilized the MovieLens 25M dataset [4]. To capture the semantic content of the items, we transformed each movie into a textual string by concatenating its title and genres. These strings were encoded using the `nomic-embed-text-v1.5` model [12], producing 768-dimensional dense vectors.

Given the dataset size relative to the model capacity, we applied Principal Component Analysis (PCA) to reduce the embedding dimensionality from 768 to 256. This reduction was essential to mitigate the risk of overfitting. The final corpus consists of 62,422

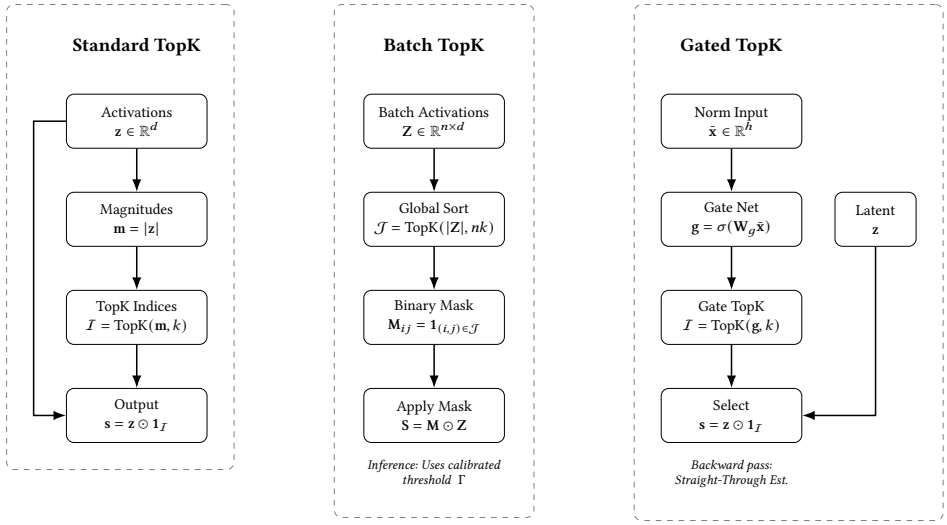

Figure 1: Comparison of sparsification operators. Left: Standard TopK selects based on local magnitude. Center: Batch TopK selects global top-$(n \times k)$ activations across the batch, allowing variable sparsity per sample. Right: Gated TopK uses a separate learnable network to predict feature importance from the input.

embeddings, which were partitioned into training (56,180), validation (3,121), and test (3,121) sets.

The models were trained using the Adam optimizer with a learning rate of 10^{-3} and a batch size of 1024. The training objective was to minimize the cosine distance between the original input embedding $\mathbf{x}$ and the reconstructed embedding $\hat{\mathbf{x}}$,

$$\mathcal{L}(\mathbf{x}, \hat{\mathbf{x}}) = 1 - \cos(\mathbf{x}, \hat{\mathbf{x}}) = 1 - \frac{\mathbf{x}^\top \hat{\mathbf{x}}}{\|\mathbf{x}\|_2 \, \|\hat{\mathbf{x}}\|_2}.$$

Training was conducted for a maximum of 25 epochs, with an early stopping mechanism (patience of 5 epochs) monitoring the validation loss.

We evaluated the CompresSAE architecture with a fixed latent dimension of $d = 1024$. The three sparsification operators—Standard TopK, Batch TopK, and Gated TopK—were tested across a range of sparsity budgets $k \in \{4, 8, 16, 32, 64\}$. All experiments were performed on a laptop equipped with an AMD Ryzen 5 5600H (6 cores, 3.3 GHz). The average training time was approximately 10 minutes per model. Table 1 summarizes the experimental configuration.

To approximate downstream retrieval utility, we report Recall@100. This metric calculates the intersection between the set of 100 nearest neighbors in the original dense space and the corresponding set in the latent or reconstructed space, ranked by cosine similarity. Additionally, we monitor reconstruction cosine loss on training and validation sets to assess model fitting. To evaluate sparsity utilization, we track the number of inactive features—latent dimensions that remain zero across the entire training set.

In Figure 2, we present the training and validation loss curves for each sparsification operator and each sparsity budget. The Standard TopK and Batch TopK operators exhibit stable convergence, with Batch TopK achieving slightly higher both training and validation loss, indicating worse generalization. On the other hand, Recall@100 on the validation set reveals that Batch TopK outperforms Standard TopK across lower budgets, suggesting that adaptive allocation of latent features benefits retrieval performance. Further, the number of inactive features (features never activated across the training set) remains low for both Standard TopK and Batch TopK, indicating effective utilization of the latent space. In contrast, the Gated TopK operator shows slower convergence and higher final loss, suggesting challenges in training stability. Additionally, Gated TopK tends to have a higher number of inactive features, indicating dead neuron issues.

Table 1: Experiment Configuration Summary

Parameter	Value
Dataset	MovieLens 25M
Encoder	nomic-embed-text-v1.5 + PCA
Embedding Dim (h)	256
Loss Function	Reconstruction Cosine Similarity
Optimizer	Adam
Learning Rate	10^{-3}
Batch Size	1024
Epochs	25
Early Stopping	5 (Patience)
Latent Dim (d)	1024
Budget (k)	{4, 8, 16, 32, 64}

Recall@100 and cosine reconstruction loss on the test set for different sparsification operators are shown in Figure 3. Batch TopK is comparable to Standard TopK and achieves slightly better results compared to Standard TopK at lower sparsity budgets ($k = 4, 8, 16$). This suggests that allowing variable sparsity per sample enables better allocation of latent features for retrieval tasks. The Gated TopK operator underperforms in both metrics, likely due to training difficulties and dead neuron issues.

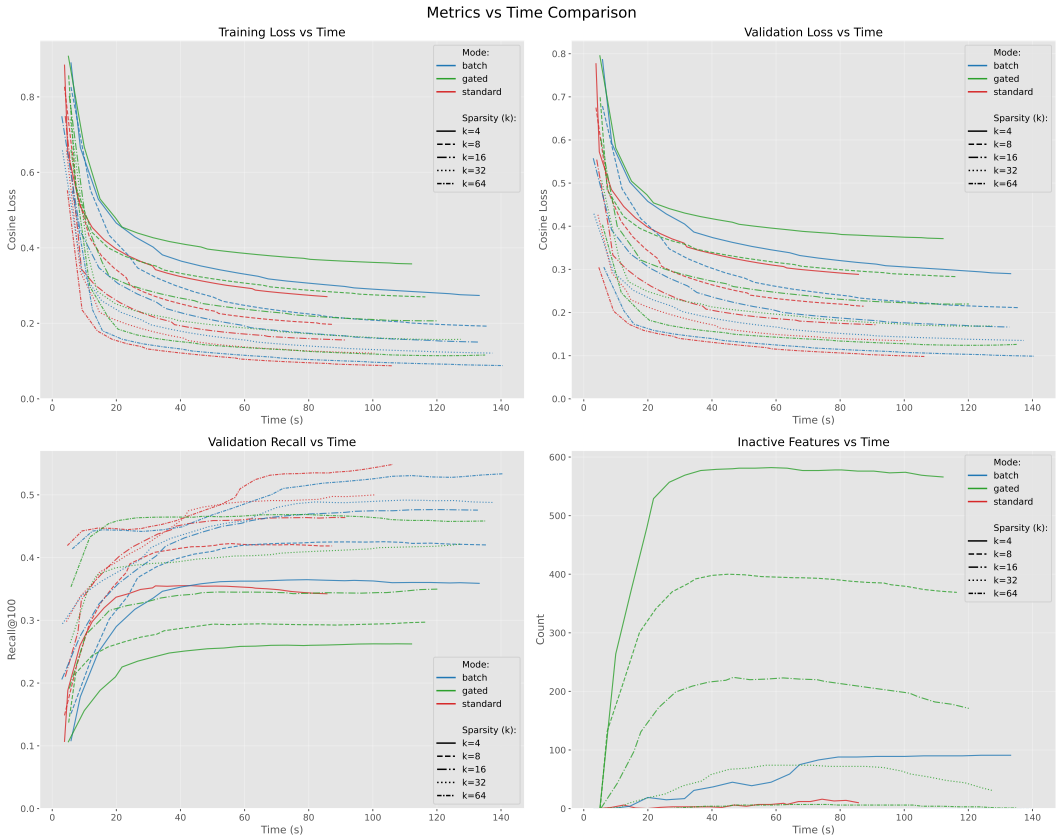

Figure 2: Training metrics for different sparsification operators.

The inference time per batch for each sparsification operator is presented in Figure 4. Batch TopK consistently demonstrates the lowest inference time across all sparsity budgets, benefiting from the fixed thresholding mechanism that avoids sorting operations during inference. Standard TopK shows higher latency due to the per-sample sorting requirement. Gated TopK has the highest inference time, attributed to the additional computations from the gating network. Figure further illustrates the actual average sparsity achieved by Batch TopK on the test set, which closely aligns, with the target budget k (and is even lower), validating the effectiveness of the threshold calibration procedure.

Finally, Table 2 presents a comprehensive comparison of the three sparsification operators across various sparsity budgets. Batch TopK achieves superior or competitive recall compared to the Standard TopK baseline while offering faster inference times due to global thresholding. Gated TopK generally underperforms in reconstruction quality and reports higher latency. The training time for Batch TopK is significantly longer (approximately 1.5x) than Standard TopK due to batch-level operations, while Gated TopK requires additional time for the gating network.

5 CONCLUSION & FUTURE WORK

In this work, we presented a comparative analysis of three sparsification operators for sparse embedding compression within the CompresSAE framework. We evaluated the Standard TopK, Batch TopK, and Gated TopK operators on the MovieLens dataset across multiple sparsity budgets. Our experiments reveal that the Batch TopK operator is a superior alternative to the baseline, offering comparable or improved retrieval performance—particularly at lower sparsity budgets—while significantly reducing inference latency due to its threshold-based selection mechanism, though this comes at the expense of longer training durations. Conversely, the gradient-based Gated TopK operator struggled with training stability and dead neurons, leading to suboptimal reconstruction and retrieval results.

These findings highlight the importance of flexible sparsity allocation in embedding compression. Future work will focus on scaling these evaluations to industry-level datasets and conducting online A/B tests to validate real-world performance. We also plan to explore the generalization of these operators to other modalities, such as images and search queries. Furthermore, investigating hardware-aware structured sparsity and improving the training stability of gated mechanisms remain promising directions for future research.

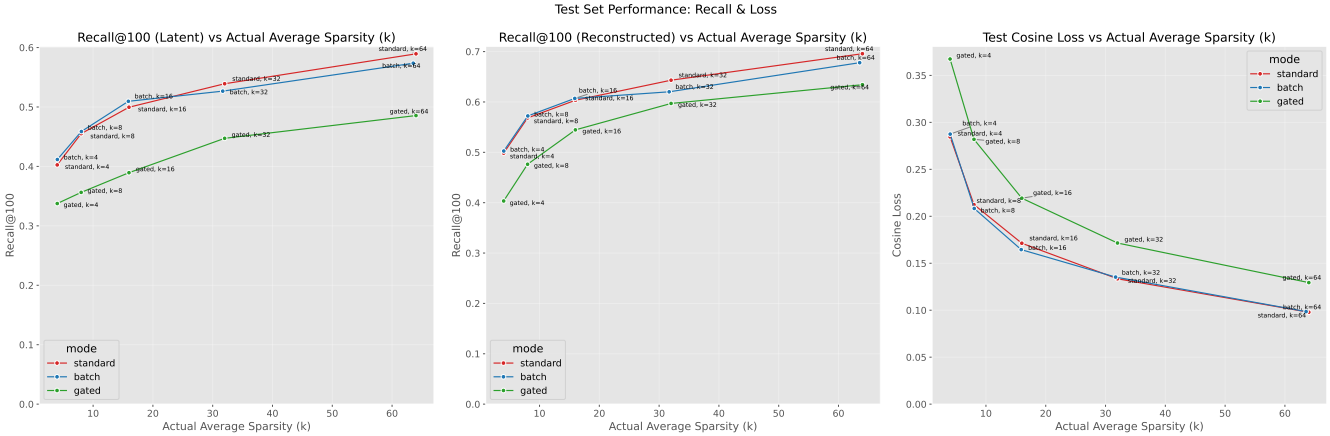

Figure 3: Test set Recall@100 and Cosine Loss for different sparsification operators.

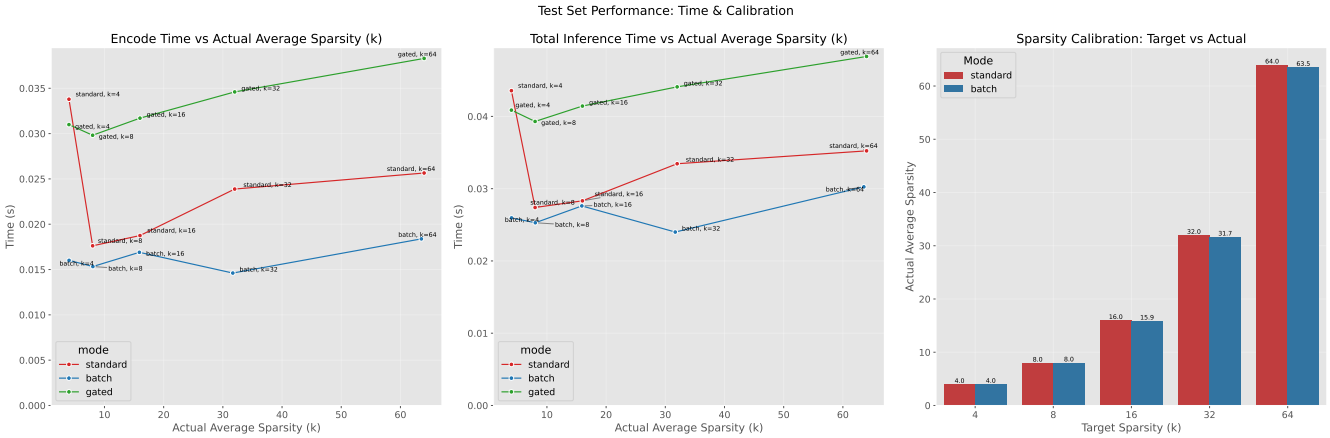

Figure 4: Test set Inference time for different sparsification operators and Average sparsity for Batch TopK.

Table 2: Performance comparison of sparsification operators across sparsity budgets (k).

Budget (k)	Operator	Actual k (avg)	Test Loss (Cosine ↓)	Test Recall (@100 ↑)	Rec. Recall (@100 ↑)	Infer. Time (ms/batch)	Train Time (s)
4	Standard	4.00	**0.285**	0.403	0.498	43.6	**85.7**
	Batch (Ours)	4.02	0.287	**0.412**	**0.503**	**26.0**	133.2
	Gated (Ours)	4.00	0.367	0.338	0.403	40.9	112.1
8	Standard	8.00	0.212	0.455	0.568	27.4	**87.3**
	Batch (Ours)	8.04	**0.208**	**0.459**	**0.572**	**25.3**	135.8
	Gated (Ours)	8.00	0.282	0.356	0.476	39.3	116.4
16	Standard	16.00	0.171	0.500	0.603	28.3	**91.4**
	Batch (Ours)	15.90	**0.165**	**0.510**	**0.607**	**27.6**	132.8
	Gated (Ours)	16.00	0.219	0.389	0.545	41.4	120.1
32	Standard	32.00	**0.133**	**0.539**	**0.643**	33.5	**100.5**
	Batch (Ours)	31.68	0.135	0.527	0.620	**24.0**	137.3
	Gated (Ours)	32.00	0.172	0.447	0.597	44.1	127.4
64	Standard	64.00	**0.098**	**0.589**	**0.696**	35.2	**106.3**
	Batch (Ours)	63.54	0.099	0.574	0.678	**30.3**	140.5
	Gated (Ours)	64.00	0.129	0.486	0.634	48.3	135.1

REFERENCES

[1] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. 2013. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432* (2013).

[2] Bart Bussmann, Jan Leike, et al. 2024. BatchTopK Sparse Autoencoders. *arXiv preprint arXiv:2412.06410* (2024).

[3] Antonio Ginart et al. 2021. Mixed Dimension Embeddings with Minimum-Variance Quantization. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*.

[4] F Maxwell Harper and Joseph A Konstan. 2015. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TiiS)* 5, 4 (2015), 1–19.

[5] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.

[6] Petr Kasalický, Martin Spišák, Vojtěch Vančura, Daniel Bohuněk, Rodrigo Alves, and Pavel Kordík. 2025. The Future is Sparse: Embedding Compression for Scalable Retrieval in Recommender Systems. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems (RecSys '25)*. Association for Computing Machinery, New York, NY, USA, 1099–1103. https://doi.org/10.1145/3705328.3748147

[7] Aditya Kusupati et al. 2022. Matryoshka representation learning. In *Advances in Neural Information Processing Systems*, Vol. 35. 30233–30249.

[8] Steinberg Li et al. 2023. Embedding Compression with Hashing for Efficient Recommendation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. (Representative citation for embedding memory bottlenecks).

[9] Alireza Makhzani and Brendan Frey. 2014. k-Sparse Autoencoders. In *International Conference on Learning Representations (ICLR)*.

[10] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G Azzolini, et al. 2019. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091* (2019). https://arxiv.org/abs/1906.00091

[11] S. Rajamanoharan et al. 2024. Jumping Ahead: Improving Reconstruction Fidelity with JumpReLU Sparse Autoencoders. *arXiv preprint arXiv:2407.14435* (2024).

[12] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.

[13] Yang Wang et al. 2024. Embedding Compression in Recommender Systems: A Survey. *arXiv preprint arXiv:2408.02304* (2024).